AI Forward Deployed Engineer

The AI Architect & FDE Course



Week 2 Day 1

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer — AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Build agents with LangChain and choose the right model for a task
- [x] Explain what's involved in Harness Engineering
- [x] Deliver an agent harness for a business goal

By the end of today, you'll be able to:

- [ ] Avoid the common traps with agent architecture
- [ ] Describe evaluations with common eval metrics
- [ ] Explain the role of observability
- [ ] Run experiments and observe results in LangFuse

Recap

What makes up an Agent Harness



Context
Prompts,
tools, memory



Workflow
The loop
the agent runs



State
Carried
between steps



Sub-agents
Delegation
& teams



Permissions
Sandboxes
and approvals



Observability
Traces
and evaluations

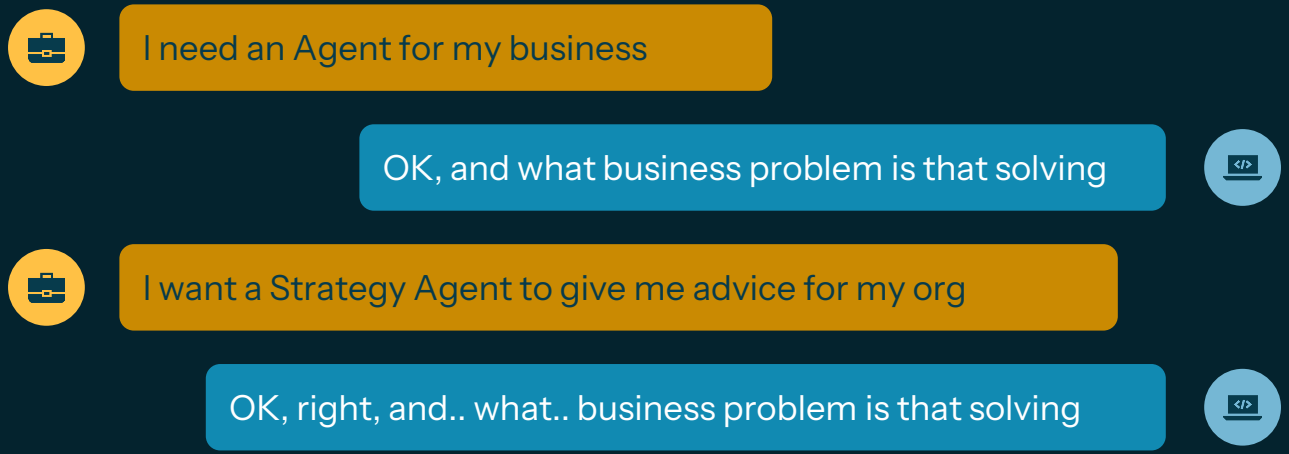


Self-improvement
And recursive
self-improvement

Today we go deeper into sub-agents and observability

Trap 1: The Agent for X

A conversation that keeps happening

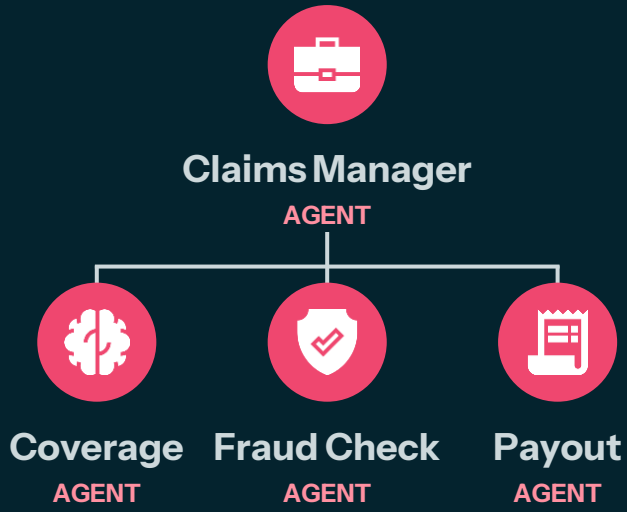


Start with a business problem you can measure.
First try solving it without Agents and without LLMs; they're not always needed.

Trap 2: human-like agents

An agent architecture with human roles and responsibilities

THE RED FLAG



THE ALTERNATIVE

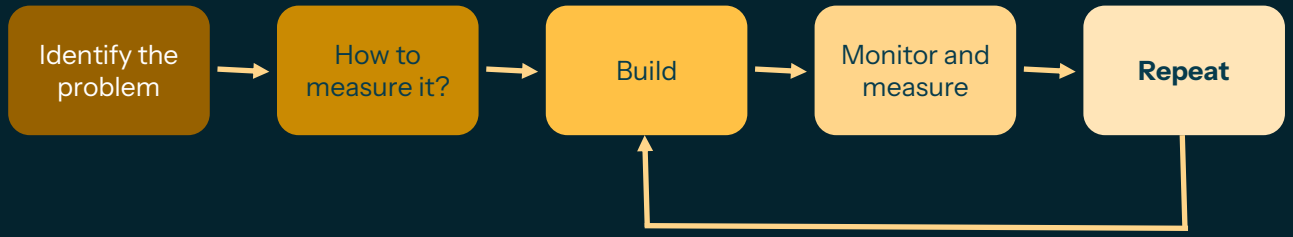
Start with one agent
one controlled workflow

Measure performance
the number from trap 1

Then add agents
when they improve performance

Trap 3: plausible vs accurate

LLMs generate plausible content.
Not necessarily accurate content.
Measure accuracy with evaluations.



The UnderCover Agent may write a believable rationale

Measure its decisions against real-world outcomes

When to add another agent

Sub-agents and agent architecture should be driven by outcomes



Context isolation for more accuracy
A focused prompt for accurate outcomes and reduced context pollution



An independent check
A focused prompt that evaluates against a rubric; potentially a different LLM



Cost efficiency
A cheap model for routine steps and a strong one for the hard reasoning



Speed optimization
Parallel lookups such as multiple research threads



Increased security with focused permissions
Tightly scoped permissions to reduce risks

An agent isn’t always the answer

Business logic often belongs in code rather than prompts



The model makes the judgment calls

Covered or not
Reads the claim against the wording

Which benefit
Such as baggage or trip delay

Which lines count
The covered items and expenses



The payout tool applies the rules

The limits
Plan limits and per-item limits

The deductible
Taken once per claim

The result
The payout and its calculation

You wouldn’t typically leave an LLM to figure out the maths behind deductibles

My recommendation

Prioritize data science and system design over agent architecture



Prioritize

Data Science concerns

Data quality
Evals
Observability



Prioritize

System Design concerns

Integrations
Context layer
API and MCP gateways
Governance



Start simple

Agent Architecture concerns

Roles and responsibilities
Design patterns
Agent communication

Start simple and add agents for the reasons we covered
Avoid building "Agent for X" just for the sake of agents
It's good to have common architecture for building agents, but stay grounded on the problem
***Coming in Week 3**: the system design of agentic AI, like Gateways and Context Layer*

Three kinds of evals

Ways to check an agent's output



Direct measurement

Based on a curated test dataset or based on real-world outcomes



LLM as a judge

A second model grades the output against specific criteria



Human review

Experts grade a sample of the agent's work by hand

Good evals tie directly to a commercial outcome - such as, revenue!
But these can be noisy; many factors outside your control go into revenue
Often best to have a set of metrics, combining model metrics and outcome metrics

The confusion matrix

For a yes or no decision like paying a claim

Actual \ Predicted	Positive	Negative
Positive	**True positive** TP	**False negative** FN
Negative	**False positive** FP	**True negative** TN

Precision

`TP / (TP + FP)`

Of those predicted positive, the share actually positive

Recall

`TP / (TP + FN)`

Of the actual positives, the share predicted positive

UnderCover tolerates false positives more than false negatives

UnderCover's confusion matrix is 3x3

The handler decided \ The agent decided	Approve	Decline	Refer
Approve	$0	$1,500	$50
Decline	Paid	$0	$50
Refer	Paid	$1,500	$0

Wrongful decline
$1,500 for complaints and lost customers

Wrongful payment
The amount paid out

Needless referral
$50 of a handler's time

Right decision
$0, or the difference on a wrong amount

A wrongful decline costs the most because it damages our reputation and customer loyalty

Scoring a run

The numbers we compare for each version of the agent



Accuracy and decision accuracy
The right decision and payout. Decision accuracy checks only the decision



Cohen's kappa
Agreement with the handlers after allowing for chance



Cost of mistakes
The cost matrix added up over every claim



Straight-through rate
The share of claims settled without a handler



Precision and recall
Approval is the positive class in the 2x2 matrix

Approving every claim gets 56% of decisions right and a kappa of 0

Observability with Langfuse

An open-source platform for tracing and evaluating LLM apps



Traces

Every model call and tool call for a claim, with tokens, cost and time



Datasets

The harder claims stored with the handlers' answers



Experiments

A version of the agent run over a dataset



Scores

Recorded for each claim and for each run

It's also powerful to surface observability directly in your business platform, for the appropriate user role (like admins) and tie it in-platform to business outcomes

Running an experiment

The four parameters to know in `run_experiment`

```
result = langfuse.run_experiment(
    name="Baseline",
    data=items,
    task=baseline_task,
    evaluators=[score_claim],
    run_evaluators=[score_run],
    max_concurrency=1,
)
print(result.format())
```

`data`
The dataset items, each a claim with the handler's answer

`task`
A function that gets one item and runs the agent on it

`evaluators`
Functions like `score_claim` that score each claim

`run_evaluators`
Functions like `score_run` that score the whole run

Langfuse keeps each experiment as a run on the dataset

Today's lab

In `labs/week2day1.ipynb` and its free twin

1. **The harder claims**
   25 claims with long calculations and tempting mistakes

2. **Tracing the agent with Langfuse**
   Set up Langfuse and trace one claim from start to finish

3. **Scoring an agent**
   Confusion and cost matrices, precision and recall, kappa and straight-through rate

4. **Experiments in Langfuse**
   The baseline on GPT-6 Luna, then the same agent with a payout tool

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



40% Complete